5. Docker 개념 및 실습

도커(Docker)는 **컨테이너**를 사용하여 애플리케이션의 개발, 배포, 실행을 위한 환경을 구축하고 관리하는 **오픈 소스 플랫폼**입니다. 애플리케이션과 그 실행에 필요한 모든 환경(코드, 런타임, 시스템 도구, 라이브러리 등)을 **가볍고 격리된 컨테이너**로 패키징하여, 어떤 환경(개발, 테스트, 운영 서버, 클라우드 등)에서든 **일관되게** 실행되도록 보장합니다.

---

1. 도커의 핵심 구성 요소

도커를 이해하는 데 필수적인 세 가지 주요 개념이 있습니다.

① 도커 이미지 (Docker Image)

- **정의:** 컨테이너를 생성(실행)하는 데 사용되는 **읽기 전용 템플릿**입니다.
- **포함 요소:** 애플리케이션을 실행하는 데 필요한 코드, 런타임, 시스템 도구, 라이브러리 및 설정 등 모든 것을 포함합니다.
- **특징:** 여러 개의 **레이어(Layer)**로 구성되어 있어, 변경 사항이 있을 때 전체를 다시 다운로드하지 않고 변경된 레이어만 다운로드합니다.
- **생성:** 주로 **Dockerfile**이라는 스크립트 파일을 사용하여 정의하고 빌드합니다.

② 도커 컨테이너 (Docker Container)

- **정의:** 도커 **이미지**를 실행한 **실행 가능한 인스턴스**입니다.
- **특징:** 호스트 OS의 커널을 공유하며, 다른 컨테이너 및 호스트 환경과 **격리된 독립적인 공간**에서 실행됩니다.
- **역할:** 애플리케이션이 구동되는 환경을 완벽하게 캡슐화하고 격리합니다. 컨테이너를 삭제해도 원본 이미지는 변경되지 않습니다.

③ 도커 레지스트리 (Docker Registry)

- **정의:** 도커 이미지를 저장하고 공유하는 중앙 저장소입니다.
- **예시: Docker Hub**가 가장 대표적인 공개 레지스트리이며, 사설(Private) 레지스트리를 구축할 수도 있습니다.
- **작동:** `docker pull` 명령으로 이미지를 다운로드하고, `docker push` 명령으로 이미지를 업로드합니다.

---

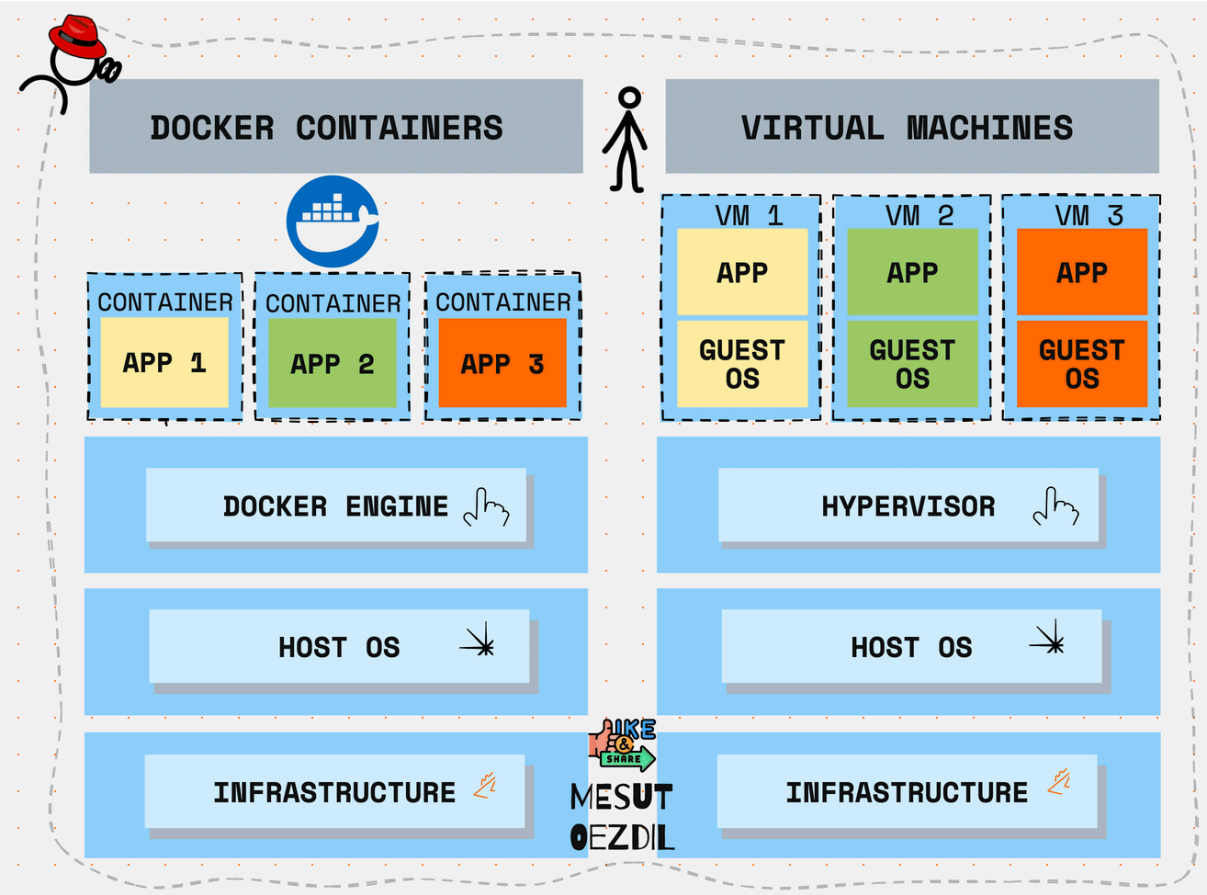
2. 도커를 사용하는 이유 (장점)

이점	상세 설명
환경 일관성 (Consistency)	개발, 테스트, 운영 환경에 관계없이 도커 이미지를 사용하면 항상 **동일한 환경**을 보장하여 "내 컴퓨터에서는 되는데..." 문제를 해결합니다.
빠른 배포 및 확장 (Speed & Scale)	컨테이너는 가볍고 시작 시간이 빨라, 새로운 서버에 애플리케이션을 배포하거나 확장(Scale-out)하는 것이 매우 신속하고 효율적입니다.
격리 (Isolation)	각 컨테이너는 독립적으로 실행되므로, 하나의 컨테이너에서 문제가 발생해도 다른 컨테이너나 호스트 시스템에 영향을 주지 않습니다.
자원 효율성 (Resource Efficiency)	VM처럼 Guest OS 전체를 포함하지 않아, VM 대비 훨씬 가볍고 호스트 시스템의 CPU, 메모리 자원을 효율적으로 사용합니다.

3. 컨테이너 vs. 가상 머신 (VM) 비교

도커의 기반이 되는 컨테이너 기술은 기존의 가상화 기술인 가상 머신(VM)과 자주 비교됩니다.

구분	도커 컨테이너 (Container)	가상 머신 (Virtual Machine, VM)
가상화 수준	**OS 레벨 가상화** (운영체제 커널 공유)	**하드웨어 레벨 가상화** (하이퍼바이저 사용)
운영 체제	**Guest OS 없음.** 호스트 OS의 커널 공유	**Guest OS 포함** (OS, 커널, 부팅 모두 포함)
무게/크기	**가벼움** (보통 수십 MB), 시작 속도 빠름	**무거움** (보통 GB 단위), 부팅 시간 필요
자원 사용	호스트 자원을 효율적으로 공유하여 사용	OS마다 자원을 할당받아 사용 (자원 소비 多)
용도	마이크로서비스, CI/CD, 애플리케이션 이식성 확보	OS 종류가 다른 환경을 분리, 완벽한 격리 필요 시



4. 도커 명령어 및 활용

4.1. 이미지 관리 (Image Management)

도커 이미지를 검색, 다운로드, 목록 확인, 삭제하는 명령어입니다.

명령어	활용	설명
`docker search <검색어>`	`docker search jenkins`	도커 허브에서 이미지 검색
`docker pull <이미지 이름>[:태그]`	`docker pull ubuntu:latest`	레지스트리에서 이미지 다운로드 (태그 생략 시 `:latest` 적용)
`docker images` 또는 `docker image ls`	`docker images`	로컬 시스템의 이미지 목록 출력
`docker rmi <이미지 ID 또는 이름>`	`docker rmi ubuntu:latest`	로컬 이미지 삭제
`docker build -t <이름> .`	`docker build -t my-web-app .`	**Dockerfile**을 사용하여 이미지 빌드 (`-t` 는 태그 지정, `.` 은 Dockerfile 위치)

4.2. 컨테이너 실행 및 관리 (Container Execution & Management)

이미지를 기반으로 컨테이너를 생성 및 실행하고, 상태를 확인하며 관리하는 명령어입니다.

명령어	활용	설명
`docker run [옵션] <이미지 이름>`	`docker run -d -p 80:80 nginx`	새 컨테이너를 **생성 및 실행**
`docker ps`	`docker ps -a`	현재 **실행 중인** 컨테이너 목록 출력 (`-a` 옵션은 종료된 컨테이너 포함)
`docker stop <ID 또는 이름>`	`docker stop my-container`	실행 중인 컨테이너 **정지** (Graceful shutdown)
`docker start <ID 또는 이름>`	`docker start my-container`	정지된 컨테이너 **시작**
`docker rm <ID 또는 이름>`	`docker rm my-container`	정지된 컨테이너 **삭제**

4.3. 컨테이너 상호작용 (Container Interaction)

실행 중인 컨테이너에 접근하거나 로그를 확인하는 명령어입니다.

명령어	활용	설명
`docker logs <ID 또는 이름>`	`docker logs -f my-container`	컨테이너의 표준 출력/오류 로그 확인 (`-f` 옵션은 실시간 출력)
`docker exec [옵션] <ID 또는 이름> <명령어>`	`docker exec -it my-container /bin/bash`	실행 중인 컨테이너에서 명령어 실행 (쉘 접속 시 `-it` 사용)
`docker cp <경로> <ID>:<경로>`	`docker cp index.html webserver:/usr/share/nginx/html`	호스트와 컨테이너 간 파일 복사
`docker inspect <ID 또는 이름>`	`docker inspect my-container`	컨테이너 또는 이미지의 상세 정보(JSON 형식) 출력

---

5. 핵심 `docker run` 옵션 활용

`docker run` 명령어는 가장 자주 사용되며 다양한 옵션을 통해 컨테이너 실행 환경을 설정합니다.

- `d` (**Detached mode**): 컨테이너를 **백그라운드**에서 실행하고 즉시 터미널 제어권을 반환합니다.
- `p <호스트 포트>:<컨테이너 포트>` (**Publish port**): 호스트 포트와 컨테이너 포트를 연결(포트 포워딩)합니다.
  - 예: `p 8080:80` (호스트의 8080 포트를 컨테이너의 80 포트로 연결)
- `-name <이름>`: 컨테이너에 **사용자 지정 이름**을 부여합니다. 이름 지정 시 ID 대신 이름을 사용하여 컨테이너를 관리할 수 있습니다.
  - 예: `-name my-web-server`
- `v <호스트 경로>:<컨테이너 경로>` (**Volume mount**): 호스트의 디렉토리 또는 볼륨을 컨테이너 내부에 마운트하여 **영구적인 데이터 저장**을 가능하게 합니다.
  - 예: `v /host/data:/container/data`

5-1. Docker 설치 및 실습